

# Assignment 2: CNN for Image Classification

**Title (Use of AI / copied code / Gen code is strictly prohibited- strict Plagiarism policies)**

Building Convolutional Neural Networks for CIFAR-10 Image Classification

## **Problem Statement**

Design, implement, and optimize a Convolutional Neural Network (CNN) using PyTorch or Keras to classify images from the CIFAR-10 dataset. You will explore various CNN architectures, regularization techniques, and data augmentation strategies to achieve high classification accuracy. This assignment emphasizes practical deep learning skills and understanding of CNN components.

## **Dataset**

### **CIFAR-10 Dataset**

- **Source:** [CIFAR-10 Official](#)
- **PyTorch:** torchvision.datasets.CIFAR10
- **Keras:** tensorflow.keras.datasets.cifar10
- **Description:** 60,000 32×32 color images in 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), 50,000 training + 10,000 test images

## **Detailed Instructions**

### **Part 1: Data Exploration & Preprocessing (10%)**

1. Load and visualize sample images from each class (10×10 grid)
2. Analyze pixel value distributions and class balance
3. Implement data preprocessing:
  - Normalize pixel values to [0, 1] or standardize to mean=0, std=1
  - Convert labels to one-hot encoding
4. Create data loaders with batch size 64
5. Implement data augmentation:
  - Random horizontal flip
  - Random rotation (±15 degrees)
  - Random crop and padding

- Color jittering (optional)

## **Part 2: Model Architecture Design (35%)**

**Task 2.1: Baseline CNN** Build a simple CNN with the following architecture: (can visualize network via ai to understand the pictorial view)

[Note- All 2 points/layers breakdown is in continuation]

Conv2D(32, 3×3) → ReLU → MaxPool(2×2) → Conv2D(64, 3×3) → ReLU → MaxPool(2×2) → Flatten → Dense(128) → ReLU → Dropout(0.5) → Dense(10) → Softmax

## **Task 2.2: Advanced CNN (Choose one)**

- **Option A:** Implement a mini-VGGNet (3-4 conv blocks with increasing filters)
- **Option B:** Implement a ResNet-inspired architecture with skip connections
- **Option C:** Design your own architecture with batch normalization

## **Task 2.3: Model Summary**

- Print model architecture with parameter counts
- Calculate total trainable parameters
- Visualize feature maps from first convolutional layer

## **Part 3: Training Strategy (25%)**

1. **Loss Function:** Categorical cross-entropy
2. **Optimizer:** SGD with momentum (0.9) or Adam
3. **Learning Rate Schedule:**
  - Start with lr=0.001
  - Reduce on plateau (factor=0.5, patience=5)
  - Or use cosine annealing
4. **Training Configuration:**
  - Epochs: 50-100
  - Early stopping (patience=15)
  - Model checkpointing (save best model)
5. **Regularization:**
  - Dropout (0.3-0.5)
  - L2 weight decay (1e-4)
  - Batch normalization

#### **Part 4: Evaluation & Analysis (20%)**

1. Plot training/validation loss and accuracy curves
2. Generate confusion matrix on test set
3. Visualize misclassified images with true/predicted labels
4. Compute per-class accuracy
5. Perform error analysis: identify which classes are most confused
6. Visualize learned filters from first conv layer
7. Generate GradCAM heatmaps for sample predictions

#### **Part 5: Advanced Experiments (10%)**

1. **Transfer Learning:** Fine-tune a pre-trained model (ResNet18, MobileNetV2)
2. **Ensemble Method:** Combine predictions from 3 different models
3. **Test-Time Augmentation:** Average predictions over augmented test samples

#### **Expected Outputs**

##### **1. Code Deliverables**

- Complete training script with modular functions
- Model definition file
- Evaluation and visualization notebook
- Requirements.txt with dependencies

##### **2. Model Files**

- Best model weights (.pth or .h5)
- Training history (JSON or CSV)

##### **3. Visualizations (minimum 8 plots)**

- Sample images from each class
- Training/validation loss curves
- Training/validation accuracy curves
- Confusion matrix
- Per-class accuracy bar chart
- Filter visualizations (first layer)
- Feature map visualizations
- GradCAM heatmaps for 10 sample images

- Misclassified images grid

#### 4. Performance Report (4-5 pages)

- Architecture design rationale
- Training methodology and hyperparameter choices
- Results comparison (baseline vs. advanced model)
- Error analysis with insights
- Discussion of overfitting/underfitting
- Suggestions for improvement

### **Findings/Observations to Document**

1. How does data augmentation affect model generalization?
2. Compare training time and accuracy: batch normalization vs. dropout
3. Which object classes are most frequently confused and why?
4. How do skip connections (ResNet) improve gradient flow?
5. What is the effect of learning rate schedule on convergence?
6. How much does transfer learning improve over training from scratch?
7. Analyze the trade-off between model complexity and performance

### **Evaluation Rubric (100 points)**

| Criteria                     | Points | Description                                                             |
|------------------------------|--------|-------------------------------------------------------------------------|
| <b>Code Implementation</b>   | 35     | Correct CNN architecture, proper training loop, efficient data pipeline |
| <b>Model Performance</b>     | 25     | Test accuracy (70%: 15pts, 75%: 20pts, 80%+: 25pts)                     |
| <b>Experimentation</b>       | 15     | Baseline vs. advanced model comparison, hyperparameter tuning           |
| <b>Visualizations</b>        | 10     | All required plots, clear and informative                               |
| <b>Analysis &amp; Report</b> | 10     | Insightful error analysis, architectural justification                  |
| <b>Code Quality</b>          | 5      | Clean, documented, reproducible code                                    |

### **Bonus Points (10)**

- Implement and compare 3+ different architectures (+3)
- Achieve >82% test accuracy (+3)
- Implement GradCAM or other interpretability method (+2)
- Deploy model as web app (Streamlit/Gradio) (+2)

### **Target Metrics**

- **Minimum:** 70% test accuracy
- **Good:** 75-80% test accuracy
- **Excellent:** >80% test accuracy

\*\*\*\*\*